

ASSIGNMENT -- INDUSTRY-ORIENTED PYTHON APPLICATION DEVELOPMENT

CSA0801 / PROGRAMMING IN PYTHON

PARTICULAR DETAILS

Course Code & Name	CSA0801 / PROGRAMMING IN PYTHON
Program	BTech
Student Name	S R Anusika
Register Number	192026008
Assignment Title	Online Food Order Management and Analysis System
Date of Issue	
Date of Submission	
Maximum Marks	100

COURSE OUTCOMES

CO1: Apply syntax and semantics of Python programming to develop unstructured coding using the interpreter, values, types, variables, expressions, functions, and modules.

CO2: Analyse control flow constructs, string operations, and functions to perform string manipulations.

CO3: Implement Python data structures such as lists, tuples, dictionaries, and sets to store, access, and manipulate unstructured data effectively.

ASSESSMENT PROFILE DETAILS

Bloom's Taxonomy Level	L3 -- Apply / L4 -- Analyse
SDG Mapping	SDG 8 -- Decent Work and Economic Growth SDG 9 -- Industry, Innovation and Infrastructure SDG 12 -- Responsible Consumption and Production
Industry / Societal Relevance	Food-delivery platforms must efficiently manage and analyze restaurant, customer, and order data to reduce delivery delays, minimize order cancellations, and improve service quality and economic sustainability for restaurant partners.

1. Assignment Problem / Challenge

Industry Case Study: Online Food Order Management and Analysis System

A large food-delivery platform partnering with over 500 restaurants and serving thousands of customers daily is struggling to manage and analyze restaurant, food item, customer, and order data effectively. The platform currently relies on fragmented spreadsheets and manual coordination between restaurant partners, delivery staff, and the support team. This approach has led to several problems:

1. Data Inconsistency: Different restaurants use different formats for storing food item names, prices, and categories.
2. Inefficient Search: Finding a specific customer's order history or a restaurant's food items takes significant time.
3. No Order Analysis: The platform cannot easily identify:
 - Customers who are high-value repeat customers
 - Restaurants with consistently low average order value
 - Food items / restaurants with high cancellation rates
4. Manual Report Generation: Creating daily/weekly order and restaurant-performance reports requires significant manual effort.
5. Data Redundancy: Customer, restaurant, and food item information is duplicated across multiple spreadsheets.
6. Error-Prone Calculations: Manual calculation of order totals, discounts, delivery charges, and cancellation percentages leads to frequent billing errors.

You are appointed as a Python Developer to design and develop a prototype "Online Food Order Management and Analysis System" that addresses these challenges. The system must model customers, restaurants, food items, and orders using appropriate Python data structures. It must efficiently store, retrieve, and analyze order data to generate meaningful insights.

CORE CHALLENGE

Design, implement, and evaluate a Python-based, menu-driven online food order management system that can efficiently store, retrieve, analyze, and report order and restaurant-performance data. The solution must demonstrate effective use of Python fundamentals, control flow, string operations, functions, and appropriate data structures to solve a real food-delivery order-management problem.

2. Problem Statement

Develop a Python-based Online Food Order Management and Analysis System with the following major modules:

Module	Key Requirements
Customer Management	Add, update, delete, and search customer details (Customer ID, Name, Phone, Address, Location). Use dictionaries for efficient storage and retrieval. Apply functions for each operation.
Restaurant Management	Add, update, delete, and search restaurant details (Restaurant ID, Name, Cuisine Category, Rating). Use dictionaries for efficient storage and retrieval.
Food Item Management	Add, update, delete, and search food item details (Food Code, Food Name, Price, Restaurant ID, Category). Use dictionaries for efficient storage and retrieval.
Order Management	Create and update orders (Pending, Confirmed, Preparing, Out for Delivery, Delivered, Cancelled) for customers against one or more food items. Store orders using appropriate data structures (nested dictionaries or lists). Apply discounts and delivery charges while computing order totals.
Order Analysis	Perform the following analyses: - Calculate Order Total (with discount and delivery charge) for a specific order - Compare two orders based on total amount (custom operation) - Identify high-value customers (based on cumulative order value) - Calculate the average order value for a specific restaurant - Find restaurants / food items with the highest cancellation rates
Search & Retrieval	Search for customers by Name, Phone, or Location. Search for food items by Restaurant or Category. Retrieve all orders for a specific customer.
Report Generation	Generate simple text-based reports: - Customer order summary (including all food items and order total) - Restaurant performance report (average order value and cancellation rate by restaurant) - Formatted order report (custom operation) showing itemized bill, discount, delivery charge, and final total
Data Persistence	Save all data (customers, restaurants, food items, orders) to CSV files for future use. Load data from CSV files at program startup.
Exception Handling	Handle invalid Customer IDs, Food Codes, Order inputs, and File Not Found errors gracefully. Use try-except-finally blocks. Implement at least one custom exception (e.g., InvalidOrderStatusError).

3. Requirements and Constraints

3.1 Functional Requirements

- Create and manage customer records.
- Create and manage restaurant records.
- Create and manage food item records.
- Create and update customer orders.
- Search and retrieve customer, restaurant, and food item information.
- Calculate order totals (with discounts and delivery charges) and other performance metrics.

- Compare orders and generate performance reports.
- Save data to CSV files.
- Load data from CSV files.
- Handle invalid operations without crashing the application.

3.2 Technical Constraints

- Use Python 3.x as the implementation language.
- Apply Python fundamentals: variables, data types, operators, and expressions.
- Use control flow constructs (if-elif-else, for, while) to guide the menu-driven program logic.
- Create functions with parameters and return values.
- Employ Python data structures:
 - Dictionaries: For storing customers (Customer ID as key), restaurants (Restaurant ID as key), food items (Food Code as key), and orders (Order ID as key).
 - Lists: For storing collections of order items, food-item lists, or customer order history.
 - Tuples: For immutable data like food item (price, category) or customer (latitude, longitude) location.
 - Sets: For unique cuisine categories or unique food codes.
- Demonstrate string manipulation (parsing customer names/addresses, formatting order-report output).
- Use file handling to read from and write to CSV files.
- Implement exception handling (try, except, finally).
- Create at least one custom exception.
- Implement at least two meaningful custom operations (e.g., compare_orders(), generate_order_report()).
- Provide evidence through screenshots, test results, and execution outputs.

3.3 Performance & Reliability Constraints

- The program should handle at least 500 customer/order records efficiently.
- No invalid data should crash the program.
- Order total calculations should be accurate and consistent.

3.4 Security & Ethical Considerations

- Data Privacy: Ensure customer information (phone, address) is not exposed inappropriately.
- Data Integrity: Prevent accidental deletion or modification of records.
- Accuracy: Ensure order total, discount, and delivery-charge calculations are mathematically correct.
- Service Consistency: Ensure all restaurants and orders are evaluated using the same status and pricing rules.

4. Customer Work & Assessment Components

Assessment Component	Marks	Expected Evidence / Output
1. Python Fundamentals Application (CO1)	10	Correct use of variables, data types, operators, and expressions; functions with parameters and return values.
2. Control Flow, String Operations & Functions (CO2)	15	Proper if-elif-else / loops for menu-driven logic; string manipulation for formatting output; well-structured functions.
3. Functions & Modular Design	10	Clear separation of concerns across modules (customer, restaurant, food item, order, and report functions).
4. Data Structures -- Lists & Tuples (CO3)	15	Correct use of lists for order-item collections and tuples for immutable price/location data.
5. Data Structures -- Dictionaries & Sets (CO3)	15	Efficient dictionary-based storage and retrieval for customers, restaurants, food items, and orders; sets for unique categories/codes.

6. Menu-Driven Functionality, Correctness & Report Generation	25	End-to-end working menu system; accurate order-total / discount / delivery-charge calculations; compare_orders() and generate_order_report() custom operations; restaurant-performance report.
7. Reflection: Design Decisions, SDG Relevance & Learning Outcomes	10	Justified design choices; explicit SDG 8/9/12 connection; honest account of challenges faced and learning gained.

5. Application of Food Item Knowledge

CO1 -- Python Fundamentals

- Variables, expressions, and statements.
- Function definition and use with parameters and return values.
- Flow of execution and function scope.
- Modules and functions (import statements).
- Understanding of the Python interpreter and interactive mode.
- Basic illustrative programs:
 - Calculate order total with discount and delivery charge
 - Calculate distance between customer and restaurant (for delivery estimation)
 - Generate a unique Order ID

CO2 -- Control Flow, Functions, Strings

- Conditional statements (if, elif, else) for order-status validation.
- Loops (for, while) for iterating through customer, restaurant, and order data.
- String operations (formatting customer names, addresses, generating order-report text).
- String methods (.strip(), .upper(), .title(), .find(), .replace()).
- Function composition.
- Illustrative programs: Linear search for customers, search for food items by restaurant, string formatting for the itemized bill.

CO3 -- Data Structures

- Lists: Storing order-item lists, customer order history, and restaurant food-item lists.
- Tuples: Storing immutable data like food item (price, category) or customer (latitude, longitude).
- Dictionaries: Storing customers (key: Customer ID), restaurants (key: Restaurant ID), food items (key: Food Code), and orders (key: Order ID).
- Sets: For unique cuisine categories or unique food codes.
- List comprehension: For efficient data processing (e.g., filtering cancelled orders).
- Aliasing and cloning: Understanding when to copy order data.
- Dictionary operations and methods: .get(), .keys(), .values(), .items().

6. Solution / Design / Methodology

6.1 Data Structure Design

Entity	Data Structure	Example	Justification
Customers	Dictionary	customers = {customer_id: {'name': 'Ravi', 'phone': '9876543210', 'address': '...', 'location': (12.9, 77.6)}}	Fast lookup by Customer ID
Restaurants	Dictionary	restaurants = {restaurant_id: {'name': 'Spice Hub', 'cuisine': 'North Indian', 'rating': 4.3}}	Fast lookup by Restaurant ID

Food Items	Dictionary	food_items = {food_code: {'name': 'Paneer Butter Masala', 'price': 220, 'restaurant_id': 'R101', 'category': 'Main Course'}}	Fast lookup by Food Code
Orders	Nested Dictionary	orders = {order_id: {'customer_id': 'C101', 'items': [('F101', 2)], 'status': 'Confirmed', 'discount': 10, 'delivery_charge': 30}}	Efficient storage and retrieval
Cuisine Categories	Set	cuisine_categories = {'North Indian', 'South Indian', 'Chinese', 'Italian'}	Unique cuisine category list

6.2 System Architecture

RECOMMENDED FLOW

Main Menu → Customer Management → Restaurant Management → Food Item Management → Order Management → Analysis → Reports → File Operations

6.3 Algorithm / Pseudocode

Order Total Calculation Algorithm:

```
FUNCTION calculate_order_total(order_id):
    IF order_id NOT IN orders:
        RETURN None
    subtotal = 0
    FOR each (food_code, quantity) IN orders[order_id]['items']:
        price = food_items[food_code]['price']
        subtotal = subtotal + (price * quantity)
    discount_amount = subtotal * (orders[order_id]['discount'] / 100)
    total = subtotal - discount_amount + orders[order_id]['delivery_charge']
    RETURN total
```

Compare Orders Algorithm (Custom Operation):

```
FUNCTION compare_orders(order_id1, order_id2):
    total1 = calculate_order_total(order_id1)
    total2 = calculate_order_total(order_id2)
    IF total1 > total2:
        RETURN order_id1 + ' has a higher order total'
    ELIF total2 > total1:
        RETURN order_id2 + ' has a higher order total'
    ELSE:
        RETURN 'Both orders have equal totals'
```

Customer Search Algorithm:

```
FUNCTION search_customers_by_name(name):
    results = []
    FOR customer_id, details IN customers.items():
        IF name.lower() IN details['name'].lower():
            results.append(customer_id)
    RETURN results
```

6.4 Module Structure

```
project_root/
├── customer_manager.py    # Customer CRUD operations
├── restaurant_manager.py  # Restaurant CRUD operations
├── fooditem_manager.py    # Food Item CRUD operations
├── order_manager.py       # Order creation and status management
├── analytics.py           # Order analytics functions (incl. compare_orders)
├── reports.py             # Order and restaurant report generation
├── file_handler.py        # CSV read/write operations
├── main.py                # Main menu and program flow
├── data/
│   └── customers.csv
```

```
├── restaurants.csv
├── fooditems.csv
└── orders.csv
```

7. Modern Tools & Evidence

- IDE: VS Code with Python extension, PyCharm, or IDLE.
- Version Control: Git and GitHub repository.
- Core Libraries: csv, datetime, statistics.
- Testing: Manual testing with test cases or simple assertions.
- Debugging: Using Python's built-in pdb or IDE debugger.

Provide evidence of tool usage through:

- Source-code screenshots.
- Execution screenshots.
- Test execution outputs.
- Git / version-control evidence.

8. Results, Testing & Validation

8.1 Minimum Test Scenarios

Test Case	Scenario	Expected Result
TC1	Add a new customer with valid details.	Customer is added successfully.
TC2	Add a new restaurant and food item with valid details.	Restaurant / Food Item is added successfully.
TC3	Create an order for a customer with multiple food items.	Order is stored successfully with the correct itemized total.
TC4	Calculate order total for an order with a discount and delivery charge.	Correct final order total is returned.
TC5	Compare two orders using compare_orders().	Correct comparison result (higher / equal) is returned.
TC6	Try to assign an invalid order status (e.g., 'Shipped').	Program raises the custom exception and handles it gracefully.
TC7	Try to load a missing CSV file at startup.	Program handles FileNotFoundError gracefully.

8.2 Order Analysis

Metric	100 Orders	500 Orders	1000 Orders
Customer Search (by name)			
Order Total Calculation Time			
Food Item Lookup Time			

9. Analysis, Trade-offs & Justification

9.1 Data Structure Choice

- Why use a dictionary for customers/restaurants/food items instead of a list?
- When would a list be more appropriate?
- Why use nested dictionaries for orders?

9.2 Function Design

- Why separate functions for each operation?
- What are the benefits of modular design?

- How do parameters and return values help?

9.3 Exception Handling

- How does your exception handling improve program reliability?
- What exceptions are most likely to occur (invalid status, missing food code, file not found)?

9.4 Control Flow

- How did you handle the main program loop?
- What conditional statements did you use for validating order status?
- How did loops help with report generation across all restaurants?

9.5 Final Recommendation

Which architecture would you recommend for a production-grade food-delivery system, and why? Support your decision with evidence.

10. Broader Considerations / Professional Responsibility

10.1 Ethical and Societal Risks

Risk	Description	Mitigation Strategy
Customer Privacy	Customer phone numbers and delivery addresses could be misused.	Implement access controls. Avoid exposing full addresses in reports. Use anonymized data for analysis where possible.
Order Accuracy	Incorrect order-total calculations could affect customer billing and restaurant payouts.	Thoroughly test all mathematical functions. Implement validation checks. Maintain an audit trail of status changes.
Service Consistency	Inconsistent status-handling or pricing practices could disadvantage some customers or restaurants.	Standardize order-status assignment rules. Ensure all restaurants follow the same discount and delivery-charge rules.
Order Data Loss	Accidental deletion of customer or order records could cause significant issues.	Implement backup mechanisms. Confirm before deletion. Save data to files regularly.

10.2 Professional Responsibility

- Data Integrity: Ensure data is accurate, consistent, and reliable.
- Transparency: Document all assumptions and methodologies.
- Reliability: Handle failures gracefully without data loss.
- Accessibility: Ensure the system is usable by all stakeholders (customers, restaurant staff, delivery partners).

11. Conclusion

Summarize the proposed system, major design decisions, Python concepts applied (CO1-CO3), achievement of requirements, major limitations, and possible future improvements.

Sample Conclusion Structure:

- System Summary: "This project successfully developed an Online Food Order Management and Analysis System..."
- Design Decisions: "Key design decisions included using dictionaries for fast retrieval, modular functions for separation of concerns, and CSV files for data persistence..."
- Food Item Knowledge Application: "The solution demonstrated CO1 through functions and modules, CO2 through data validation and string operations, and CO3 through appropriate data structure selection..."

- Results: "The system successfully managed 500 customer and order records, calculated order totals accurately, and generated meaningful restaurant-performance reports..."
- Limitations: "The current prototype does not support concurrent access or a GUI interface..."
- Future Improvements: "Future work could include database integration, a web interface, and advanced analytics such as delivery-time prediction..."

12. Customer Reflection

1. What did you learn from this assignment beyond what was directly taught in the classroom?
 - Write about your experience with real-world data management challenges.
 - Reflect on the importance of data structure selection.
2. Which Python concept was most challenging to integrate and why?
 - Was it data structures? Exception handling? File handling?
 - Explain the challenges and how you solved them.
3. What problem did you encounter while implementing the system, and how did you solve it?
 - Provide specific examples (e.g., handling duplicate entries, searching customers).
4. How did your understanding of data structures influence the design of your system?
 - Describe how you chose dictionaries, lists, tuples, and sets.
5. If you were given additional time/resources, what would you improve and why?
 - Interactive interface? Database integration? Advanced analytics (delivery-time prediction)?

13. References

1. Python Documentation. (2026). Python 3.x Library Reference. <https://docs.python.org/3/>
2. Downey, A. B. (2016). Think Python (2nd ed.). O'Reilly Media.
3. Severance, C. (2016). Python for Everybody.
4. Van Rossum, G., & Drake, F. L. (2011). The Python Language Reference Manual.
5. AI-assisted tools used: [e.g., GitHub Copilot, ChatGPT for debugging and documentation].

14. Assessment Rubric -- 100 Marks

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Apply syntax and semantics of Python programming (CO1)	10	Correctly identifies all entities and workflow from the problem statement; data types, operators, and control structures used efficiently and correctly throughout.	Identifies most entities/workflow with minor gaps; mostly correct use of data types, operators, and control structures.	Basic entities/workflow identified; data types and control structures used but with some inefficiencies or errors.	Requirements misunderstood; frequent misuse of data types/operators/control flow affecting correctness.
Analyse control flow constructs, string operations, and functions (CO2)	15	Menu-driven control flow, string formatting, and functions are well designed with appropriate validation and clean modular scoping throughout.	Reasonable control flow and string handling with minor structural or validation gaps.	Control flow and string handling exist but are weak (e.g., excessive duplication, limited validation).	Little to no meaningful control flow or string handling; logic not properly organized into functions.

Functions and string manipulations (CO2)	10	Functions correctly used for shared/reusable logic; string manipulation (formatting, parsing) is accurate and consistently applied across the report and order modules.	Functions and string handling used with minor logical errors.	Functions or string handling implemented but underused or partially correct.	Functions and/or string manipulation missing or non-functional.
Implement Python data structures -- lists, tuples (CO3)	15	Clear and efficient use of lists for order/item collections and tuples for immutable data (price, location); design choices are justified.	Lists and tuples correctly implemented but adaptation/justification is limited to one or two cases.	Basic list/tuple usage present but shallow; some inconsistency in application.	No meaningful or correct use of lists or tuples.
Dictionaries and sets to store, access, and manipulate data (CO3)	15	Dictionaries and sets implemented correctly and efficiently across all entities (customers, restaurants, food items, orders); at least two meaningful custom operations implemented correctly.	Most dictionary/set usage correct; at least one custom operation implemented correctly with minor issues.	Only some dictionary/set usage or one custom operation implemented; incomplete.	Dictionaries/sets and/or custom operations are missing, incorrect, or cause errors.
Menu-Driven Functionality, Correctness & Report Generation	25	All features (registration, order creation/update, total computation, comparison, report generation) work correctly end-to-end via a clean menu-driven interface.	Most features work correctly; minor bugs in edge cases or report formatting.	Core features work but some functionality (e.g., report or total calculation) is incomplete or buggy.	Program does not run reliably or major features are missing/non-functional.
Reflection: Design Decisions, SDG Relevance & Learning Outcomes	10	Thoughtful justification of design choices, clear connection to SDG 8/9/12 relevance, and honest, specific account of challenges faced and learning gained.	General justification of design choices; SDG connection and challenges mentioned but lack depth.	Reflection present but generic; limited justification of design or account of learning outcomes.	No genuine reflection submitted, or content is generic with no real design/SDG/learning connection.

15. CO -- PO -- Assessment Mapping

Assessment Component	CO	Bloom's Level	Marks
Python Fundamentals Application	CO1	L3 -- Apply	10
Control Flow, Strings & Functions	CO2	L4 -- Analyze	15
Functions & Modular Design	CO2	L3/L4 -- Apply/Analyze	10
Data Structures -- Lists & Tuples	CO3	L4 -- Analyze	15
Data Structures -- Dictionaries & Sets	CO3	L4/L5 -- Analyze/Evaluate	15
Menu-Driven Functionality & Report Generation	CO1, CO2, CO3	L5 -- Evaluate	25

Reflection & Broader Considerations	CO1, CO2, CO3	L4 -- Analyze	10
TOTAL	100		

16. Expected Customer Deliverables

1. Working Python application (multiple .py files)
2. Source code with comments
3. System architecture diagram or flowchart
4. Algorithms / pseudocode
5. Minimum five test cases with results
6. Performance comparison for different dataset sizes
7. Screenshots / tool evidence
8. Final technical report
9. References
10. Customer reflection
11. GitHub repository upload (source code and commit history)

CENTRAL HIGHER-ORDER CHALLENGE

"Design, implement, experimentally evaluate, and justify a Python-based, menu-driven online food order management system. Using Python fundamentals, control flow, string operations, functions, and appropriate data structures, determine the most effective way to store, retrieve, and analyze customer, restaurant, and order data. Demonstrate the consequences of poor data structure choice, compare your design to alternative approaches, validate the results experimentally, and defend your final engineering decisions using evidence."

ASSIGNMENT PROFILE DETAILS

Primary Bloom's Taxonomy	L4 -- Analyze and L5 -- Evaluate
Nature of Assignment	Industry Case Study + Design + Implementation + Experimentation + Critical Evaluation
Challenge Level	Medium / Complex / Higher-Order